The classical and foundational algorithms (**Caesar, Monoalphabetic, Playfair, Hill, Transposition, RC4, RSA, and Diffie–Hellman**) **do not need external cryptographic libraries** like OpenSSL or Python's cryptography package.
They can be implemented directly using Python's built-in standard library (math, random, secrets) because their mathematical rules rely on basic modular arithmetic, array shuffling, and string manipulations.

Why Modern Ciphers Need Libraries vs. Classical Ciphers

Algorithm Group	Needs Crypto Library?	Why?
Caesar, Monoalphabetic, Playfair, Transposition	**No** (Pure Python)	Character lookups, string indexing, modular addition (\mathbb{Z}_{26}).
Hill Cipher	**No** (Pure Python or numpy)	Simple 2 \times 2 or 3 \times 3 matrix multiplication modulo 26.
RC4	**No** (Pure Python)	256-byte state array swap operations and XOR.
RSA / Diffie–Hellman	**No** (For lab/learning)	Python natively supports arbitrarily large integers and modular exponentiation: pow(base, exp, mod).
DES / AES	**Yes** (Recommended)	Computing 16 Feistel rounds or Galois Field GF(2^8) matrix mixing in pure Python is extremely slow and lacks side-channel/timing attack protections. Libraries leverage C bindings and native CPU instructions (**AES-NI**).

Pure Python Implementations (Zero External Dependencies)

Here is how each algorithm is built from scratch without installing anything.

1. Caesar Cipher (Shift n)

```
def caesar_cipher(text: str, n: int, mode='encrypt') -> str:
    shift = (n % 26) if mode == 'encrypt' else (-n % 26)
    res = []
    for ch in text:
        if ch.isupper():
            res.append(chr((ord(ch) - 65 + shift) % 26 + 65))
        elif ch.islower():
            res.append(chr((ord(ch) - 97 + shift) % 26 + 97))
        else:
            res.append(ch)
    return "".join(res)
```

2. Monoalphabetic Substitution Cipher

```
import string

STD_ALPHABET = string.ascii_uppercase
KEY_ALPHABET = "QWERTYUIOPASDFGHJKLZXCVBNM"

def monoalphabetic_cipher(text: str, key: str = KEY_ALPHABET,
mode='encrypt') -> str:
    mapping = {p: c for p, c in zip(STD_ALPHABET, key)} if mode ==
'encrypt' else {c: p for p, c in zip(STD_ALPHABET, key)}
    return "".join(mapping.get(c, c) if c.isupper() else
mapping.get(c.upper(), c.upper()).lower() if c.islower() else c for c
in text)
```

3. Columnar Transposition Cipher

```
import math

def transposition_encrypt(text: str, key: str) -> str:
    col_order = sorted(list(range(len(key))), key=lambda k: key[k])
    num_cols = len(key)
    num_rows = math.ceil(len(text) / num_cols)
    padded = text.ljust(num_rows * num_cols, 'X')

    # Read column by column in sorted key order
    grid = [padded[i * num_cols : (i + 1) * num_cols] for i in
range(num_rows)]
    return "".join("".join(grid[r][c] for r in range(num_rows)) for c
in col_order)

def transposition_decrypt(cipher: str, key: str) -> str:
    num_cols = len(key)
    num_rows = len(cipher) // num_cols
    col_order = sorted(list(range(num_cols)), key=lambda k: key[k])

    # Reconstruct columns
    grid = [[''] * num_cols for _ in range(num_rows)]
    idx = 0
    for c in col_order:
        for r in range(num_rows):
            grid[r][c] = cipher[idx]
            idx += 1
    return "".join("".join(row) for row in grid)
```

4. Playfair Cipher (5 \times 5 Grid)

```
def playfair_matrix(key: str):
    key = "".join(dict.fromkeys(key.upper().replace("J", "I") +
"ABCDEFGHIKLMNOPQRSTUVWXYZ"))
    return [list(key[i:i+5]) for i in range(0, 25, 5)]

def playfair_cipher(text: str, key: str, mode='encrypt') -> str:
    matrix = playfair_matrix(key)
    pos = {matrix[r][c]: (r, c) for r in range(5) for c in range(5)}
    shift = 1 if mode == 'encrypt' else -1

    # Clean and create digraphs
    raw = [c for c in text.upper().replace("J", "I") if c.isalpha()]
    pairs, i = [], 0
    while i < len(raw):
        a = raw[i]
        b = raw[i+1] if i + 1 < len(raw) else 'X'
        if a == b:
            pairs.append((a, 'X'))
            i += 1
        else:
            pairs.append((a, b))
            i += 2

    res = []
    for a, b in pairs:
        r1, c1 = pos[a]
        r2, c2 = pos[b]
        if r1 == r2:
            res.extend([matrix[r1][(c1 + shift) % 5], matrix[r2][(c2 +
shift) % 5]])
        elif c1 == c2:
            res.extend([matrix[(r1 + shift) % 5][c1], matrix[(r2 +
shift) % 5][c2]])
        else:
            res.extend([matrix[r1][c2], matrix[r2][c1]])
    return "".join(res)
```

5. Hill Cipher (2 \times 2 Matrix)

```
# Extended Euclidean algorithm for modular inverse
def mod_inverse(a, m=26):
    for x in range(1, m):
        if (a * x) % m == 1:
```

```
            return x
    raise ValueError("Matrix determinant not invertible modulo 26")

def hill_cipher(text: str, key_2x2=[[3, 3], [2, 5]], mode='encrypt')
-> str:
    cleaned = [c for c in text.upper() if c.isalpha()]
    if len(cleaned) % 2 != 0:
        cleaned.append('X')

    det = (key_2x2[0][0] * key_2x2[1][1] - key_2x2[0][1] *
key_2x2[1][0]) % 26
    inv_det = mod_inverse(det, 26)

    if mode == 'decrypt':
        mat = [
            [( key_2x2[1][1] * inv_det) % 26, (-key_2x2[0][1] *
inv_det) % 26],
            [(-key_2x2[1][0] * inv_det) % 26, ( key_2x2[0][0] *
inv_det) % 26]
        ]
    else:
        mat = key_2x2

    out = []
    for i in range(0, len(cleaned), 2):
        v = [ord(cleaned[i]) - 65, ord(cleaned[i+1]) - 65]
        c0 = (mat[0][0] * v[0] + mat[0][1] * v[1]) % 26
        c1 = (mat[1][0] * v[0] + mat[1][1] * v[1]) % 26
        out.extend([chr(c0 + 65), chr(c1 + 65)])
    return "".join(out)
```

6. RC4 Stream Cipher (KSA + PRGA)

```
def rc4(data: bytes, key: bytes) -> bytes:
    # 1. Key-Scheduling Algorithm (KSA)
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]

    # 2. Pseudo-Random Generation Algorithm (PRGA) & XOR
    i = j = 0
    out = bytearray()
    for byte in data:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
```

```
        S[i], S[j] = S[j], S[i]
        keystream_byte = S[(S[i] + S[j]) % 256]
        out.append(byte ^ keystream_byte)
    return bytes(out)
```

7. RSA (Key Gen, Modular Exponentiation)

```
import math

def rsa_key_gen():
    # Example prime pairs for lab demonstration (use larger primes in
practice)
    p, q = 61, 53
    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    while math.gcd(e, phi) != 1:
        e = 3
    d = pow(e, -1, phi)  # Python 3.8+ built-in modular inverse
    return (e, n), (d, n)

def rsa_encrypt(msg_int: int, pub_key: tuple) -> int:
    e, n = pub_key
    return pow(msg_int, e, n)  # Fast modular exponentiation (C = M^e
mod n)

def rsa_decrypt(cipher_int: int, priv_key: tuple) -> int:
    d, n = priv_key
    return pow(cipher_int, d, n)  # M = C^d mod n
```

8. Diffie–Hellman Key Exchange

```
def diffie_hellman():
    # Public parameters agreed upon by Alice and Bob
    p = 23   # Shared prime modulus
    g = 5    # Shared generator primitive root

    # Alice's side
    alice_private = 6
    alice_public = pow(g, alice_private, p)  # (5^6) mod 23 = 8

    # Bob's side
    bob_private = 15
    bob_public = pow(g, bob_private, p)    # (5^15) mod 23 = 19
```

```
    # Exchange public keys over insecure network:
    alice_shared = pow(bob_public, alice_private, p)   # (19^6) mod 23
= 2
    bob_shared = pow(alice_public, bob_private, p)     # (8^15) mod 23
= 2

    assert alice_shared == bob_shared
    return alice_shared
```

Summary Checklist for Your Lab Work

- **Run without installing anything:** Caesar, Monoalphabetic, Transposition, Playfair, Hill, RC4, RSA, and Diffie–Hellman.
- **Keep pip install cryptography for: DES and AES** on images and video files to ensure standard CBC padding, Initialization Vector (IV) generation, and hardware throughput measurement.

To run these algorithms across **text, numbers, images, and video** using production crypto libraries without performance bottlenecks or memory crashes, your laptop needs a setup that handles both file I/O and hardware acceleration efficiently.

1. Optimize Your Linux / Python Environment

Running this inside your Linux terminal home directory (`~/Cryptography/` rather than `/mnt/c/`) gives you native Linux POSIX filesystem performance and avoids the cross-OS filesystem translation overhead of WSL.

Inside your project directory, set up an isolated virtual environment:

cd ~/Cryptography
python3 -m venv venv
source venv/bin/activate
pip install cryptography numpy matplotlib pillow

- `cryptography`: C-bindings to OpenSSL; utilizes **AES-NI hardware acceleration** directly on your CPU.
- `numpy`: SIMD-vectorized array math for Hill Cipher and modulo-256 byte operations.
- `pillow`: Image conversion and byte-level manipulation without breaking metadata.
- `matplotlib`: Performance plotting (`matplotlib.use('Agg')` ensures it saves cleanly without needing an X11/Wayland display server).

2. How Your Laptop Converts Each Input Type to Crypto Payloads

Modern cryptographic algorithms (AES, DES, RC4) only see streams of bits/bytes (`0x00` to `0xFF`). The key is standardizing every input type into Python's `bytes` format before passing it to

the crypto library:

Input Type	Conversion to `bytes`	Recovery to Original Type
Plain Text	`text.encode('utf-8')`	`raw_bytes.decode('utf-8')`
Numbers / Integers	`num.to_bytes(length, 'big')`	`int.from_bytes(raw_bytes, 'big')`
Images (`.jpg`, `.png`)	Read raw: `open(path, 'rb').read()`	Write raw: `open(path, 'wb').write(decrypted)`
Video (`.mp4`)	Chunked stream / buffer reads	Write chunked byte buffers to output path

3. Unified Runner Script for All Input Types

Save this script as `run_crypto_lab.py`. It accepts **plain text, numbers, or image files**, runs them through the algorithm, verifies decryption integrity, and reports **time taken, latency, and throughput**.

```
import os
import time
from PIL import Image
import io

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding


# --- Standardized AES & DES Engines ---
def run_aes_cbc(payload: bytes, key: bytes = b"16ByteSecretKey!", mode: str = "encrypt") ->
bytes:
    if mode == "encrypt":
        iv = os.urandom(16)
        cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
        padder = padding.PKCS7(128).padder()
        padded = padder.update(payload) + padder.finalize()
        return iv + cipher.encryptor().update(padded) + cipher.encryptor().finalize()
    else:
        iv = payload[:16]
        ct = payload[16:]
        cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
        unpadder = padding.PKCS7(128).unpadder()
        decrypted = cipher.decryptor().update(ct) + cipher.decryptor().finalize()
        return unpadder.update(decrypted) + unpadder.finalize()
```

```
def run_des_cbc(payload: bytes, key: bytes = b"8ByteKey", mode: str = "encrypt") -> bytes:
    key_24 = key * 3  # TripleDES equivalent for standard DES block size
    if mode == "encrypt":
        iv = os.urandom(8)
        cipher = Cipher(algorithms.TripleDES(key_24), modes.CBC(iv))
        padder = padding.PKCS7(64).padder()
        padded = padder.update(payload) + padder.finalize()
        return iv + cipher.encryptor().update(padded) + cipher.encryptor().finalize()
    else:
        iv = payload[:8]
        ct = payload[8:]
        cipher = Cipher(algorithms.TripleDES(key_24), modes.CBC(iv))
        unpadder = padding.PKCS7(64).unpadder()
        decrypted = cipher.decryptor().update(ct) + cipher.decryptor().finalize()
        return unpadder.update(decrypted) + unpadder.finalize()


# --- Benchmark & Pipeline Wrapper ---
def benchmark_payload(name: str, payload: bytes, algo_fn):
    size_bytes = len(payload)
    size_kb = size_bytes / 1024.0
    size_mb = size_kb / 1024.0

    # 1. Encrypt
    t0 = time.perf_counter()
    encrypted = algo_fn(payload, mode="encrypt")
    t1 = time.perf_counter()

    enc_time = t1 - t0
    throughput = (size_mb / enc_time) if enc_time > 0 else 0.0
    latency_us_per_kb = (enc_time / size_kb * 1e6) if size_kb > 0 else 0.0

    # 2. Decrypt & Verify
    decrypted = algo_fn(encrypted, mode="decrypt")
    assert decrypted == payload, "Decryption mismatch! Data corrupted."

    print(f"\n--- Results for: {name} ---")
    print(f"Payload Size : {size_bytes} bytes ({size_mb:.4f} MB)")
    print(f"Time Taken   : {enc_time:.6f} seconds")
    print(f"Throughput   : {throughput:.2f} MB/s")
    print(f"Latency      : {latency_us_per_kb:.3f} µs/KB")
    print(f"Integrity    : [VERIFIED MATCH]")
    return encrypted, decrypted


# --- Interactive CLI ---
def main():
```

```
    print("=====================================================")
    print("      MULTI-INPUT CRYPTO LAB HARNESS (AES / DES)     ")
    print("=====================================================")
    print("Choose Input Type:")
    print("1. Text String")
    print("2. Number (Integer / Float)")
    print("3. Image File (.png / .jpg)")
    print("4. Video File (.mp4)")

    choice = input("Enter choice (1-4): ").strip()
    algo_choice = input("Select Algorithm (1 for AES-128, 2 for DES): ").strip()
    cipher_engine = run_aes_cbc if algo_choice == "1" else run_des_cbc
    engine_name = "AES-128" if algo_choice == "1" else "DES"

    if choice == "1":
        msg = input("\nEnter text: ")
        payload = msg.encode("utf-8")
        _, dec = benchmark_payload(f"Text Message via {engine_name}", payload, cipher_engine)
        print(f"Recovered Text: {dec.decode('utf-8')}")

    elif choice == "2":
        num_str = input("\nEnter number (e.g. 42 or 3.14159): ").strip()
        payload = num_str.encode("utf-8")  # String encoding handles floats and arbitrary precision
        _, dec = benchmark_payload(f"Numeric Input via {engine_name}", payload, cipher_engine)
        print(f"Recovered Number: {dec.decode('utf-8')}")

    elif choice == "3":
        img_path = input("\nEnter path to image file: ").strip()
        if not os.path.exists(img_path):
            print("File not found! Creating a synthetic test image...")
            img = Image.new('RGB', (1024, 1024), color=(73, 109, 137))
            img_path = "test_image.png"
            img.save(img_path)

        with open(img_path, "rb") as f:
            payload = f.read()

        enc, dec = benchmark_payload(f"Image File ({img_path}) via {engine_name}", payload,
cipher_engine)

        # Save output image
        out_path = "decrypted_" + os.path.basename(img_path)
        with open(out_path, "wb") as f:
            f.write(dec)
        print(f"Saved decrypted image to: {out_path}")

    elif choice == "4":
        vid_path = input("\nEnter path to video file: ").strip()
```

```
    if not os.path.exists(vid_path):
        print("Video not found! Creating synthetic 10 MB payload...")
        payload = os.urandom(10 * 1024 * 1024)
    else:
        with open(vid_path, "rb") as f:
            payload = f.read()

    enc, dec = benchmark_payload(f"Video Payload via {engine_name}", payload,
cipher_engine)
    print("Video buffer encrypted and decrypted with zero loss.")


if __name__ == "__main__":
    main()
```

4. Running Large Media on Laptops with Limited RAM

If you test large video files (> 500\text{ MB}):

- **Do not use `f.read()`** to load the entire file into memory at once, as that duplicates memory usage (3 \times file size across plaintext, padding, and ciphertext buffers).

Process in chunks: Use an 8 KB or 64 KB chunking loop:

```
cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
encryptor = cipher.encryptor()

with open("input.mp4", "rb") as fin, open("output.enc", "wb") as fout:
    fout.write(iv)
    while chunk := fin.read(64 * 1024):  # 64 KB buffer
        if len(chunk) % 16 != 0:
            # Pad final block
            padder = padding.PKCS7(128).padder()
            chunk = padder.update(chunk) + padder.finalize()
        fout.write(encryptor.update(chunk))
    fout.write(encryptor.finalize())
```

-

This keeps RAM consumption under 15\text{ MB} regardless of whether the video is 100 MB or 10 GB.